

1. Task 4: Dockerization & Deployment

Objective

The objective of this task was to containerize the Task Manager application using Docker. The application consists of a React frontend, a Node.js/Express backend, and a MongoDB database. Docker Compose was used to run all services together.

Technologies Used

- React.js
- Node.js
- Express.js
- MongoDB
- Docker
- Docker Compose

Project Architecture

The project consists of three services:

Frontend

- Built using React.js
- Provides the user interface
- Runs on Port 3000

Backend

- Built using Node.js and Express.js
- Provides REST API endpoints
- Runs on Port 5000

Database

- MongoDB container
- Stores task data
- Runs on Port 27017

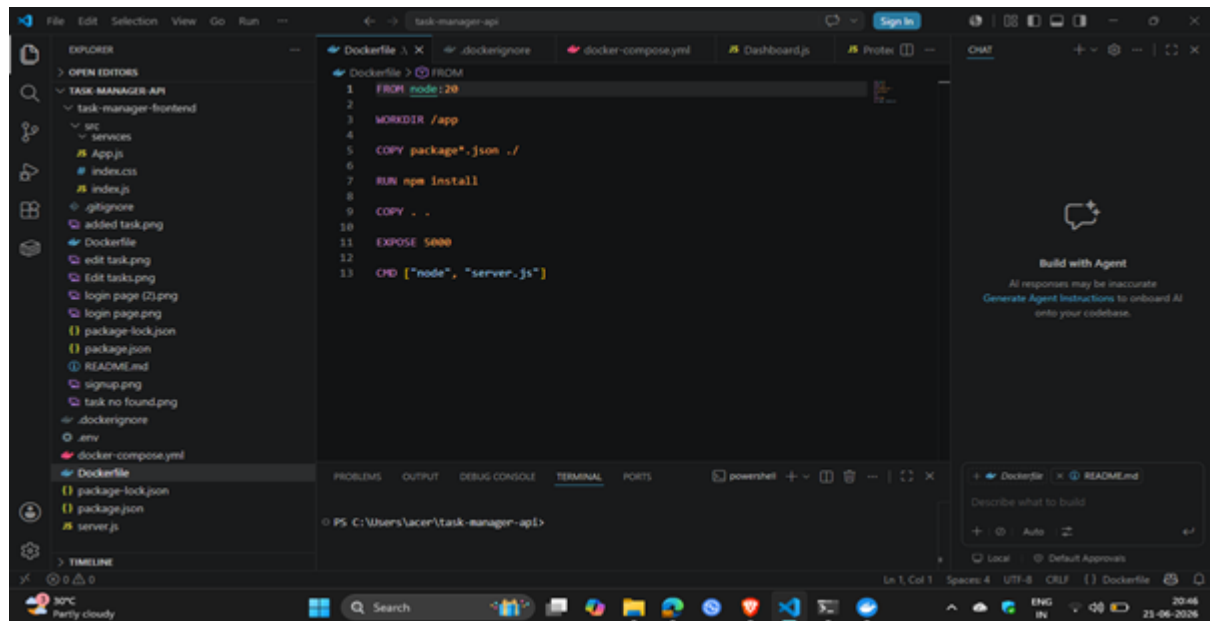
Backend Dockerfile

A Dockerfile was created for the backend application.

Purpose

- Create a Node.js container
- Install dependencies
- Run the Express server

Screenshot:



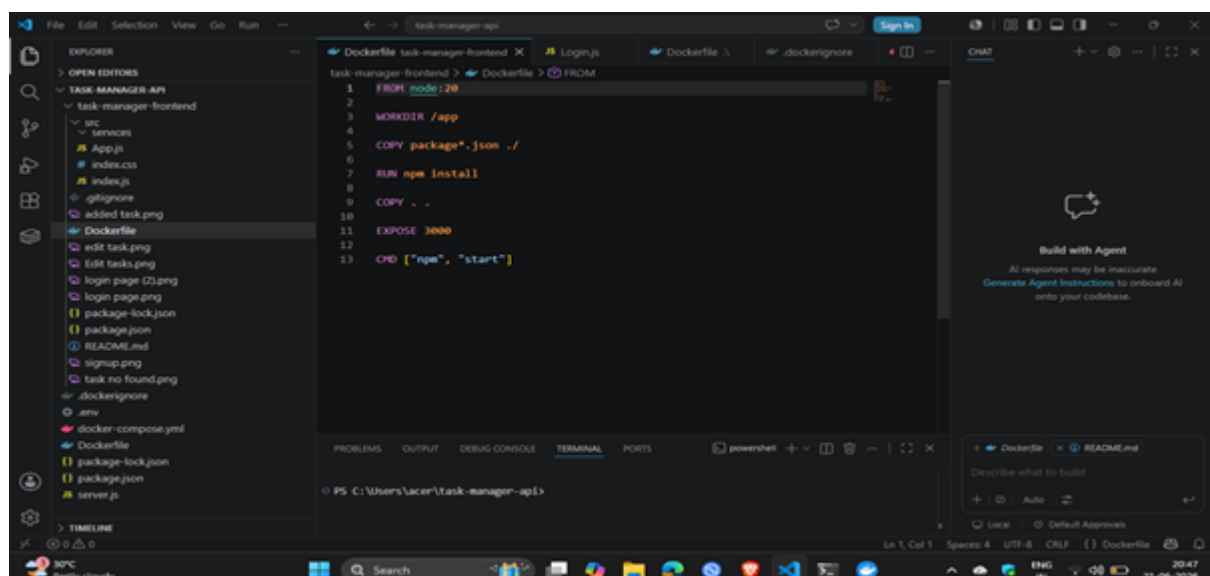
Frontend Dockerfile

A Dockerfile was created for the React frontend application.

Purpose

- Create a React container
- Install dependencies
- Start the React application

Screenshot:



Docker Compose Configuration

A docker-compose.yml file was created to manage all services.

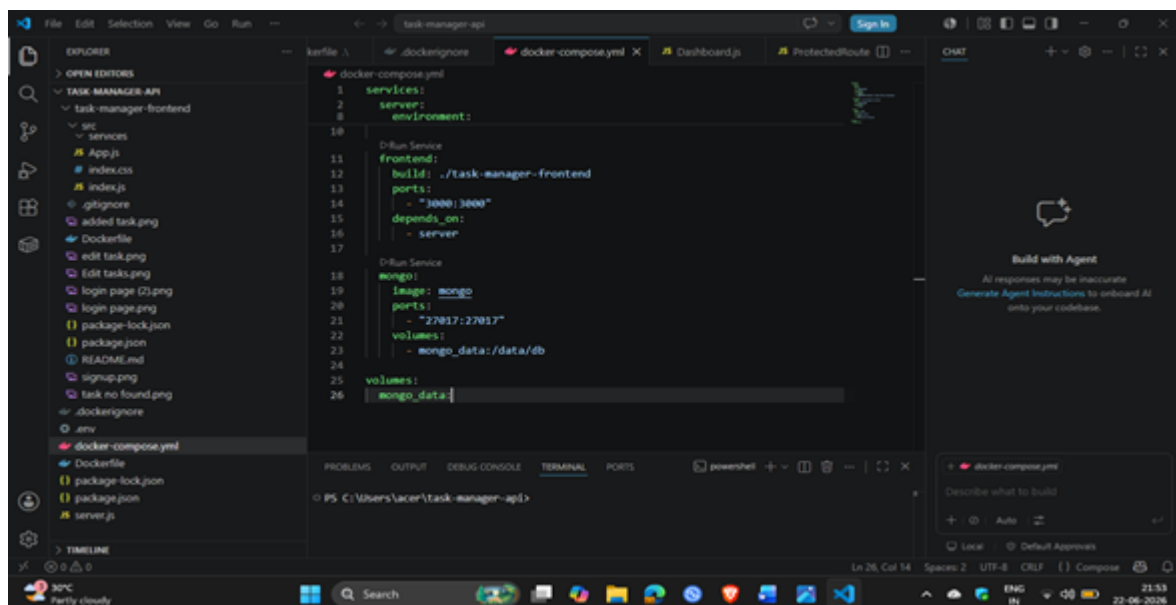
Services Included

- Frontend
- Backend
- MongoDB

Benefits

- Easy deployment
- Single command startup
- Simplified container management

Screenshot:



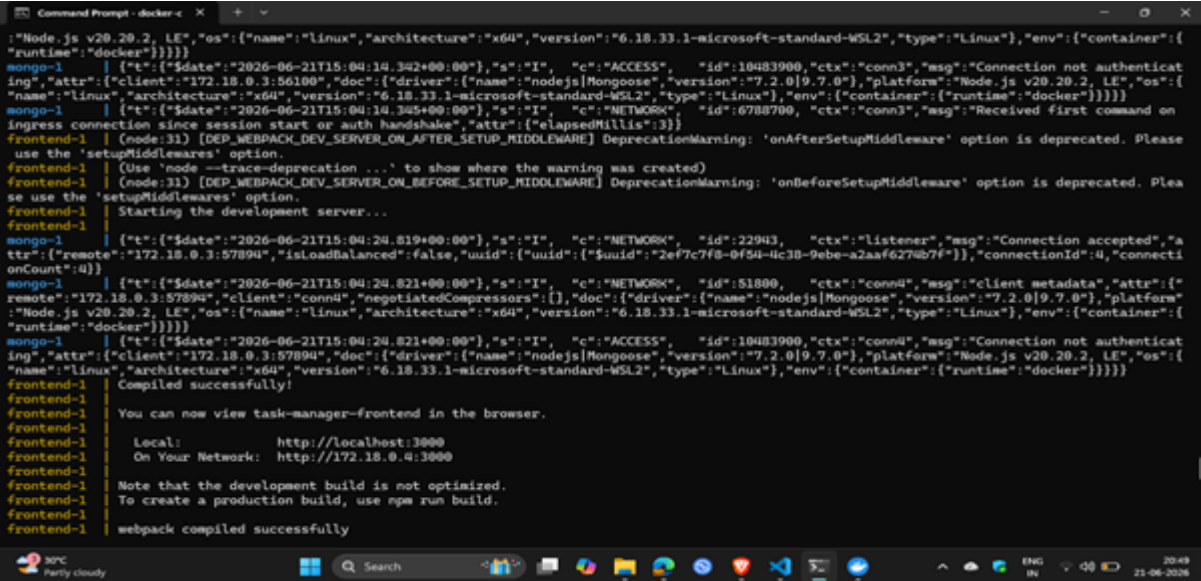
Building and Running Containers

The following command was used:

docker-compose up --build

This command builds the images and starts all containers.

Screenshot:



```
Command Prompt - docker-c
Node.js v20.2.0, LE, os: {"name": "Linux", "architecture": "x64", "version": "6.18.33.1-microsoft-standard-WSL2", "type": "Linux"}, env: {"container": {"runtime": "docker"}}}
mongo-1 | {"t":{"date":"2026-06-21T15:04:14.342+00:00"},"s":"I", "c":"ACCESS", "id":10483900, "ctx":"conn3", "msg":"Connection not authenticating", "attr":{"client":"172.18.0.3:56100", "doc":{"driver":{"name":"nodejs|Mongoose", "version":"7.2.0|9.7.0"}, "platform":"Node.js v20.2.0, LE", "os":{"name":"Linux", "architecture":"x64", "version":"6.18.33.1-microsoft-standard-WSL2", "type":"Linux"}, "env":{"container":{"runtime":"docker"}}}}}
mongo-1 | {"t":{"date":"2026-06-21T15:04:14.345+00:00"},"s":"I", "c":"NETWORK", "id":6788700, "ctx":"conn3", "msg":"Received first command on ingress connection since session start or auth handshake", "attr":{"elapsedMillis":3}}
frontend-1 | (node:31) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
frontend-1 | (node:31) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
frontend-1 | Starting the development server...
mongo-1 | {"t":{"date":"2026-06-21T15:04:24.819+00:00"},"s":"I", "c":"NETWORK", "id":22943, "ctx":"listener", "msg":"Connection accepted", "attr":{"remote":"172.18.0.3:57894", "isLoadBalanced":false, "uuid":{"uuid":"2ef7c7f8-0f5d-4c38-9e8e-a2aaf627b7f"}, "connectionId":4, "connectionCount":4}}
mongo-1 | {"t":{"date":"2026-06-21T15:04:24.821+00:00"},"s":"I", "c":"NETWORK", "id":51800, "ctx":"conn4", "msg":"client metadata", "attr":{"remote":"172.18.0.3:57894", "client":"conn4", "negotiatedCompressors":[]}, "doc":{"driver":{"name":"nodejs|Mongoose", "version":"7.2.0|9.7.0"}, "platform":"Node.js v20.2.0, LE", "os":{"name":"Linux", "architecture":"x64", "version":"6.18.33.1-microsoft-standard-WSL2", "type":"Linux"}, "env":{"container":{"runtime":"docker"}}}}}
mongo-1 | {"t":{"date":"2026-06-21T15:04:24.821+00:00"},"s":"I", "c":"ACCESS", "id":10483900, "ctx":"conn4", "msg":"Connection not authenticating", "attr":{"client":"172.18.0.3:57894", "doc":{"driver":{"name":"nodejs|Mongoose", "version":"7.2.0|9.7.0"}, "platform":"Node.js v20.2.0, LE", "os":{"name":"Linux", "architecture":"x64", "version":"6.18.33.1-microsoft-standard-WSL2", "type":"Linux"}, "env":{"container":{"runtime":"docker"}}}}}
frontend-1 | Compiled successfully!
frontend-1 | You can now view task-manager-frontend in the browser.
frontend-1 | Local: http://localhost:3000
frontend-1 | On Your Network: http://172.18.0.4:3000
frontend-1 | Note that the development build is not optimized.
frontend-1 | To create a production build, use npm run build.
frontend-1 | webpack compiled successfully
```

Running Containers

The following command was used to verify running containers:

docker ps

The output confirmed that:

- MongoDB container is running
- Backend container is running
- Frontend container is running

Backend Verification

The backend API was tested using:

http://localhost:5000

Response received:

```
{
  "message": "API is working"
}
```

This confirms that the backend service is running successfully.

Frontend Verification

The frontend application was accessed using:

<http://localhost:3000>

The React application loaded successfully and displayed the user interface.

MongoDB Connectivity Verification

The terminal logs showed:

MongoDB Connected
Server running on port 5000

This confirms successful communication between the backend and MongoDB container.

Challenges Faced

- Docker container configuration
- MongoDB connection setup
- Frontend and backend communication
- Docker Compose service linking

All issues were resolved through testing and configuration updates.

Github

<https://github.com/afrosefathima455-hub/task-manager-api.git>